



JUnit-Tests

Tim Schmitz
16.05.2026

Inhalt

Minimalkonfiguration	3
Test-Klasse	3
Test-Aufbau	3
Test-Ausführung	4
Build-Test	4

JUnit-Tests

Unit-Tests in Java werden am besten mit dem Testframework JUnit durchgeführt. Die aktuelle Version des Frameworks ist JUnit 5. Es besteht aus einer Plattform und der Test-Engine Jupiter.

Minimalkonfiguration

Für die Minimalkonfiguration werden die folgenden Java Archive-Dateien benötigt:

- junit-jupiter-api-5.6.1
- junit-platform-commons-1.6.1

Test-Klasse

Die JUnit Test-Klasse hat einen möglichst ähnlichen Namen und einen analogen Paketpfad wie die Klasse, die unter diesem Test steht. Ebenso sollten die Methoden der JUnit Test-Klasse entsprechend lautende Namen besitzen wie in dem folgenden Beispiel:

Bezeichnung	Source	JUnit Test
Klassen-Name	Property	PropertyTest
Paket	de.domain.application.util	de.domain.application.util.test
Methode	getTimPC()	testGetTimPC()

Tabelle 1: Bezeichner von Original und Test-Klasse

Test-Aufbau

Der JUnit-Test testet eine Methode oder Eigenschaft einer Klasse. Im folgenden Beispiel wird der Funktionsweise einer Methode getestet, indem sie auf eine Equals-Assertion getestet wird, also auf die Behauptung einer Gleichheit.

Zunächst müssen die verwendeten Methoden und Annotationen importiert werden sowie die Klasse, die unter Test steht:

```
import static org.junit.jupiter.api.Assertions.assertEquals;

// Notwendig wegen JUnit 5
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;

// Klasse unter Test
import de.domain.application.util.Property;
```

Anschließend wird mit der Annotation `@BeforeAll` sichergestellt, dass - bevor irgendein Testfall ausgeführt wird - die entsprechenden Daten geladen werden. Dafür werden statische Methoden der unter zu Test stehenden Klassen geladen.

```
@BeforeAll
public static void prepare() {
    Property.setProperties();
    Property.load();
}
```

Anschließend können die Testfälle definiert werden. Im Beispiel wird die Methode `getTimPC()` getestet und dabei auf einen behaupteten Erwartungswert abgefragt:

```
@Test
void testGetTimPC() {
    assertEquals(EXPECTED_TIM_PC, Property.getTimPC());
}
```

Der Test vergleicht nun den Erwartungswert EXPECTED_TIM_PC (einen String) mit dem Ergebnis der getesteten Methode Property.getTimPC().

Test-Ausführung

Der JUnit-Test kann ausgeführt werden, indem in der Projektstruktur der IDE mit der rechten Maustaste auf die Test-Klasse navigiert wird und dort Run [Testklasse]... gewählt wird. Die IDE führt dann alle Tests der Test-Klasse aus und listet die Resultate auf.

Build-Test

Wenn die JUnit-Tests in den Apache Ant Build-Prozess integriert werden sollen, dann bedeutet dies eine Zusatzarbeit, die man zumindest in der Entwicklungsumgebung IntelliJ IDEA Community Edition durchführen muss. Das Target compile-test übersetzt die Testklassen der Anwendung in ausführbare Tests. Dazu muss die Jupiter API Archive-Datei in den Klassenpfad eingebunden werden.

```
<!-- ***** -->
<!-- Target compile-test übersetzt die Testtreiber -->
<!-- ***** -->
<target name="compile-test" depends="compile">
  <javac srcdir="${project.test}" destdir="${runtime.test}" includeantruntime="false">
    <classpath>
      <pathelement location="${junit.api}/junit-jupiter-api-5.6.2.jar"/>
      <pathelement path="${runtime.build}"/>
    </classpath>
  </javac>
</target>
```

Anschließend können im Target test die übersetzten Testklassen ausgeführt werden. Wichtig ist hier allerdings die Ordnerstruktur der Entwicklungsumgebung IntelliJ richtig zu nutzen und die entsprechenden Klassen der JUnit 5-Komponentenbibliothek in das Ant-Lib-Verzeichnis von IntelliJ IDEA zu kopieren. Der Pfad dazu lautet

C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.2.5\lib\ant\lib

Die Testfälle selber liegen in der Ablaufumgebung und werden dort ausgeführt. Die Testergebnisse werden dort in ein Verzeichnis output geschrieben und liegen - wie bereits erwähnt - als XML- und textuelle Dateien vor.

```
<!-- *****+ -->
<!-- Target test führt die Tests durch -->
<!-- ACHTUNG: die jupiter- und junit-apis werden in den ant/lib-Ordner von IntelliJ kopiert -->
<!-- ***** -->
<target name="test" depends="compile-test">
  <junitlauncher>
    <classpath>
      <pathelement path="${runtime.test}"/>
      <pathelement path="${runtime.build}"/>
    </classpath>
    <testclasses outputdir="${runtime.output}">
      <fileset dir="${runtime.test}"/>
      <listener type="legacy-brief" sendSysOut="true"/>
      <listener type="legacy-xml" sendSysErr="true" sendSysOut="true"/>
    </testclasses>
  </junitlauncher>
</target>
```